GAI Disclosure (ChatGPT 5.6 Luna) - 9/24/2026

Prompt 1 (Created based on assignment requirements & simplified to allow for easier AI parsing)

Write a C or C++ program that reads a line of text and extracts a single valid IPv4 address — optionally followed by a port number. Only digits, periods (`.`), and colons (`:`) are ever part of a valid token; every other character is garbage and is skipped. A candidate token must match the address grammar **in full** — no partial matches, no truncating to find a valid piece inside a longer run. Ex. 255.255.255.255:3

An address is four octets separated by periods (octet.octet.octet.octet), each octet 1–3 digits, value 0–255, no leading zero unless the value is exactly 0. An optional `:port` may follow the fourth octet: 1–5 digits, value 0–65535, same leading-zero rule. If a colon is present, the port must be fully valid or the entire match — address included — is rejected.

Function prototype

The function returns whether a valid address was found, and delivers the address and port through two additional parameters.

C:

```
// Returns 1 if a valid address was found, 0 otherwise.
// On success: *outAddress holds the 32-bit value, and
// *outPort holds the port number, or -1 if no port was present.
// On failure: *outAddress is set to 0 and *outPort is set to -1.
int extractIPv4(const char* str, unsigned long* outAddress, int*
outPort);
```

C++

```
// Returns true if a valid address was found, false otherwise.
// On success: outAddress holds the 32-bit value,
// and outPort holds the port number, or -1 if no port was present.
// On failure: outAddress is set to 0 and outPort is set to -1.
bool extractIPv4(const std::string& str, unsigned long& outAddress,
int& outPort);
```

Requirements

- No Regex or other similar built in functions, only iteration through the string

- There is only one match, if there is more text, it is ignored
- Reject anything that doesn't match the exact format of above
- On success print Extracted IPv4 Address: A.B.C.D (Decimal value: N, port : P)
- There should be an input loop (main), you must match the function format above, and print to terminal outputs

Result

```
#include <iostream>
#include <string>

bool extractIPv4(const std::string& str, unsigned long& outAddress, int& outPort)
{
    // Default failure values
    outAddress = 0;
    outPort = -1;

    size_t i = 0;

    while (i < str.length())
    {
        // Skip garbage characters
        if (!(str[i] >= '0' && str[i] <= '9') &&
            str[i] != '.' && str[i] != ':')
        {
            i++;
            continue;
        }

        // Find the end of this candidate token.
        size_t start = i;
        while (i < str.length() &&
               ((str[i] >= '0' && str[i] <= '9') ||
                str[i] == '.' || str[i] == ':'))
        {
            i++;
        }

        size_t end = i;

        // Parse the entire candidate token.
        unsigned long octets[4] = {0, 0, 0, 0};
        int octetCount = 0;
        size_t pos = start;
```

```
bool valid = true;

// Parse exactly four octets
for (int octet = 0; octet < 4; octet++)
{
    // Must have at least one digit
    if (pos >= end || str[pos] < '0' || str[pos] > '9')
    {
        valid = false;
        break;
    }

    size_t digitStart = pos;
    unsigned long value = 0;

    // Read the digits
    while (pos < end && str[pos] >= '0' && str[pos] <= '9')
    {
        value = value * 10 + (str[pos] - '0');
        pos++;
    }

    size_t digitCount = pos - digitStart;

    // 1-3 digits only
    if (digitCount > 3)
    {
        valid = false;
        break;
    }

    // No leading zero unless the value is exactly 0
    if (digitCount > 1 && str[digitStart] == '0')
    {
        valid = false;
        break;
    }

    // Must be 0-255
    if (value > 255)
    {
        valid = false;
        break;
```

```
    }

    octets[octet] = value;
    octetCount++;

    // First three octets must have a period
    if (octet < 3)
    {
        if (pos >= end || str[pos] != '.')
        {
            valid = false;
            break;
        }

        pos++;
    }
}

// If the address itself was invalid, reject the entire token.
if (!valid || octetCount != 4)
{
    continue;
}

// Optional port
int port = -1;

if (pos < end)
{
    // If anything remains, it must begin with ':'
    if (str[pos] != ':')
    {
        continue;
    }

    pos++;

    // Port must contain at least one digit
    if (pos >= end || str[pos] < '0' || str[pos] > '9')
    {
        continue;
    }

    size_t portStart = pos;
```

```
    unsigned long portValue = 0;

    while (pos < end && str[pos] >= '0' && str[pos] <= '9')
    {
        portValue = portValue * 10 + (str[pos] - '0');
        pos++;
    }

    size_t portDigits = pos - portStart;

    // 1-5 digits
    if (portDigits > 5)
    {
        continue;
    }

    // No leading zero unless value is exactly 0
    if (portDigits > 1 && str[portStart] == '0')
    {
        continue;
    }

    // 0-65535
    if (portValue > 65535)
    {
        continue;
    }

    port = static_cast<int>(portValue);
}

// Nothing in the candidate token should remain.
if (pos != end)
{
    continue;
}

// Construct the 32-bit IPv4 value.
outAddress =
    (octets[0] << 24) |
    (octets[1] << 16) |
    (octets[2] << 8)  |
    octets[3];
```

```
            outPort = port;

            return true;
        }

        return false;
    }


    int main()
    {
        std::string input;

        while (true)
        {
            std::cout << "Enter a line of text (or type quit to exit): ";

            if (!std::getline(std::cin, input))
                break;

            if (input == "quit")
                break;

            unsigned long address;
            int port;

            if (extractIPv4(input, address, port))
            {
                // Convert the numeric address back to dotted decimal
                unsigned int a = (address >> 24) & 255;
                unsigned int b = (address >> 16) & 255;
                unsigned int c = (address >> 8) & 255;
                unsigned int d = address & 255;

                std::cout << "Extracted IPv4 Address: "
                          << a << "." << b << "." << c << "." << d
                          << " (Decimal value: " << address
                          << ", port : " << port << ")\n";
            }
            else
            {
                std::cout << "No valid IPv4 address found.\n";
            }
        }
```

```
    return 0;
}
```

Prompt 2 (Sight read through code & quick testing to find issue that was caused by simple prompt mismatch)

Port when the user sees it should say none instead of -1 when it can't find it

Output

```
#include <iostream>
#include <string>

bool extractIPv4(const std::string& str, unsigned long& outAddress, int& outPort)
{
    // Default failure values
    outAddress = 0;
    outPort = -1;

    size_t i = 0;

    while (i < str.length())
    {
        // Skip garbage characters
        if (!(str[i] >= '0' && str[i] <= '9') &&
            str[i] != '.' && str[i] != ':')
        {
            i++;
            continue;
        }

        // Find the end of this candidate token.
        size_t start = i;
        while (i < str.length() &&
               ((str[i] >= '0' && str[i] <= '9') ||
                str[i] == '.' || str[i] == ':'))
        {
            i++;
        }
```

```
size_t end = i;

// Parse the entire candidate token.
unsigned long octets[4] = {0, 0, 0, 0};
int octetCount = 0;
size_t pos = start;

bool valid = true;

// Parse exactly four octets
for (int octet = 0; octet < 4; octet++)
{
   // Must have at least one digit
   if (pos >= end || str[pos] < '0' || str[pos] > '9')
   {
      valid = false;
      break;
   }

   size_t digitStart = pos;
   unsigned long value = 0;

   // Read the digits
   while (pos < end && str[pos] >= '0' && str[pos] <= '9')
   {
      value = value * 10 + (str[pos] - '0');
      pos++;
   }

   size_t digitCount = pos - digitStart;

   // 1-3 digits only
   if (digitCount > 3)
   {
      valid = false;
      break;
   }

   // No leading zero unless the value is exactly 0
   if (digitCount > 1 && str[digitStart] == '0')
   {
      valid = false;
      break;
```

```
    }

    // Must be 0-255
    if (value > 255)
    {
        valid = false;
        break;
    }

    octets[octet] = value;
    octetCount++;

    // First three octets must have a period
    if (octet < 3)
    {
        if (pos >= end || str[pos] != '.')
        {
            valid = false;
            break;
        }

        pos++;
    }
}

// If the address itself was invalid, reject the entire token.
if (!valid || octetCount != 4)
{
    continue;
}

// Optional port
int port = -1;

if (pos < end)
{
    // If anything remains, it must begin with ':'
    if (str[pos] != ':')
    {
        continue;
    }

    pos++;
```

```
    // Port must contain at least one digit
    if (pos >= end || str[pos] < '0' || str[pos] > '9')
    {
        continue;
    }

    size_t portStart = pos;
    unsigned long portValue = 0;

    while (pos < end && str[pos] >= '0' && str[pos] <= '9')
    {
        portValue = portValue * 10 + (str[pos] - '0');
        pos++;
    }

    size_t portDigits = pos - portStart;

    // 1-5 digits
    if (portDigits > 5)
    {
        continue;
    }

    // No leading zero unless value is exactly 0
    if (portDigits > 1 && str[portStart] == '0')
    {
        continue;
    }

    // 0-65535
    if (portValue > 65535)
    {
        continue;
    }

    port = static_cast<int>(portValue);
}

// Nothing in the candidate token should remain.
if (pos != end)
{
    continue;
}
```

```
        // Construct the 32-bit IPv4 value.
        outAddress =
            (octets[0] << 24) |
            (octets[1] << 16) |
            (octets[2] << 8)  |
            octets[3];

        outPort = port;

        return true;
    }

    return false;
}


int main()
{
    std::string input;

    while (true)
    {
        std::cout << "Enter a line of text (or type quit to exit): ";

        if (!std::getline(std::cin, input))
            break;

        if (input == "quit")
            break;

        unsigned long address;
        int port;

        if (extractIPv4(input, address, port))
        {
            // Convert the numeric address back to dotted decimal
            unsigned int a = (address >> 24) & 255;
            unsigned int b = (address >> 16) & 255;
            unsigned int c = (address >> 8) & 255;
            unsigned int d = address & 255;

             std::cout << "Extracted IPv4 Address: "
              << a << "." << b << "." << c << "." << d
              << " (Decimal value: " << address
```

```
            << ", port : ";

        if (port == -1)
            std::cout << "None";
        else
            std::cout << port;

        std::cout << ")\n";
    }
    else
    {
        std::cout << "No valid IPv4 address found.\n";
    }
  }

  return 0;
}
```

Prompt 3 (Used to ensure that the prologue comment was accurate)

Can you give me a prologue comment with author (you)

Output

```
/*
 * Program: IPv4 Address Extractor
 * Author: ChatGPT
 * Date: September 24, 2026
 *
 * Description:
 * This program reads lines of text and searches for a valid IPv4 address,
 * optionally followed by a port number. Garbage characters are ignored,
 * while candidate tokens containing digits, periods, and colons must match
 * the complete IPv4 address and optional port grammar.
 *
 * The program rejects invalid octets, leading zeros, invalid ports, and
 * partial matches. On success, the IPv4 address is displayed along with
 * its decimal value and port number, or "None" if no port is present.
 */
```

Prompt 4 (Used to create more test cases out of scope and possibly break program)

can you tell me what I need to change to update format to have text like

```
Enter a string (or 'END' to quit): connecting to 192.168.1.1 now
Extracted IPv4 address: 192.168.1.1 (decimal value: 3232235777, port:
none)
Enter a string (or 'END' to quit): server=10.0.0.255:8080end
Extracted IPv4 address: 10.0.0.255 (decimal value: 167772415, port:
8080)
Enter a string (or 'END' to quit): 192a168.1.1.1
Extracted IPv4 address: 168.1.1.1 (decimal value: 2818638081, port:
none)
Enter a string (or 'END' to quit): 192.168.1.1.
Invalid input: no valid IPv4 address found
Enter a string (or 'END' to quit): Connection from 192.168.1.1 refused
Extracted IPv4 address: 192.168.1.1 (decimal value: 3232235777, port:
none)
Enter a string (or 'END' to quit): 192.168.01.1
Invalid input: no valid IPv4 address found
Enter a string (or 'END' to quit): 1.2.3.4:99999
Invalid input: no valid IPv4 address found
Enter a string (or 'END' to quit): 12.34.56
Invalid input: no valid IPv4 address found
Enter a string (or 'END' to quit): no number here
Invalid input: no valid IPv4 address found
Enter a string (or 'END' to quit): END
Program terminated.
```

Output

```
int main()
{
    std::string input;

    while (true)
    {
        std::cout << "Enter a string (or 'END' to quit): ";
```

```
        if (!std::getline(std::cin, input))
            break;

        if (input == "END")
        {
            std::cout << "Program terminated.\n";
            break;
        }

        unsigned long address;
        int port;

        if (extractIPv4(input, address, port))
        {
            // Convert the numeric address back to dotted decimal
            unsigned int a = (address >> 24) & 255;
            unsigned int b = (address >> 16) & 255;
            unsigned int c = (address >> 8) & 255;
            unsigned int d = address & 255;

            std::cout << "Extracted IPv4 address: "
                      << a << "." << b << "." << c << "." << d
                      << " (decimal value: " << address
                      << ", port: ";

            if (port == -1)
                std::cout << "none";
            else
                std::cout << port;

            std::cout << ")\n";
        }
        else
        {
            std::cout << "Invalid input: no valid IPv4 address found\n";
        }
    }

    return 0;
}
```

Test Cases

AI Prompt

Write a C or C++ program that reads a line of text and extracts a single valid IPv4 address — optionally followed by a port number — embedded anywhere in that text. Only digits, periods (`.`), and colons (`:`) are ever part of a valid token; every other character is garbage and is skipped. A candidate token must match the address grammar **in full** — no partial matches, no truncating to find a valid piece inside a longer run.

An address is four octets separated by periods (octet.octet.octet.octet), each octet 1–3 digits, value 0–255, no leading zero unless the value is exactly 0. An optional `:port` may follow the fourth octet: 1–5 digits, value 0–65535, same leading-zero rule. If a colon is present, the port must be fully valid or the entire match — address included — is rejected.

Give me 10 that will likely break my program test cases

Output

Here are 10 adversarial test cases designed to catch partial matching, leading-zero, range, and port-validation bugs:

1. `abc 192.168.1.1xyz`
    - **Expected:** `192.168.1.1`
    - Tests garbage immediately surrounding a valid address.
2. `abc 192.168.1.1:8080xyz`
    - **Expected:** `192.168.1.1:8080`
    - Tests a valid address with a valid port.
3. `abc 192.168.1.1:99999`
    - **Expected: No match**
    - Port has 5 digits but exceeds `65535`.
4. `abc 192.168.1.1:00080`
    - **Expected: No match**
    - Port has a leading zero.
5. `abc 192.168.001.1`
    - **Expected: No match**
    - Octet has a leading zero.
6. `abc 192.168.1.256`
    - **Expected: No match**
    - Last octet exceeds `255`.

7. `abc 192.168.1.999:80`
    - **Expected: No match**
    - Invalid octet followed by a valid-looking port; catches programs that try to extract a valid partial address.
8. `abc 192.168.1.1:80:90`
    - **Expected: No match**
    - Extra colon means the entire candidate token is invalid; catches programs that stop at the first valid-looking portion.
9. `abc 192.168.1.1:`
    - **Expected: No match**
    - Colon is present, so a complete valid port is required.
10. `abc 192.168.1.1.5`

- **Expected: No match**
- Five octets; catches programs that incorrectly extract `192.168.1.1` from the beginning of a longer numeric/dot token.

A particularly nasty set for your program would be **#7, #8, #9, and #10**, because they test the requirement that the grammar must match the **entire candidate token rather than just finding a valid substring**.

Changes

All of the submitted code was generated by ChatGPT. My only real contribution was directing the AI through prompt generation and reviewing the code generated. I tested it against valid and invalid inputs to ensure the final program matched the required format. Most of the prompts following the first one were only simple changes to the output format and didn't require changes to the underlying code & overall ideas which ChatGPT got on the first try. Prompt 2 was to ensure port numbers matched what they really were (-1 vs none). Prompt 3 for code comment & Prompt 4 for user input display changes. I reviewed the code and ensured that it met required grammar.

Verification

I reviewed the submitted code line by line and understood what all of it does. I tested the program against the test cases shown above created by chatGPT & those on canvas. The test cases above were generated in an attempt to break the program or test specific edge cases to ensure valid output. There are currently no known limitations for the program as it passes all expected input.